

Self-Evolving Security Framework for LLM Agents via Verifiable Guardrail Tools

First Author^{1,2*}, Second Author^{2,3†} and Third Author^{1,2†}

^{1*}Department, Organization, Street, City, 100190, State, Country.

²Department, Organization, Street, City, 10587, State, Country.

³Department, Organization, Street, City, 610101, State, Country.

*Corresponding author(s). E-mail(s): iauthor@gmail.com;
Contributing authors: iiauthor@gmail.com; iiiauthor@gmail.com;

[†]These authors contributed equally to this work.

Abstract

With capabilities expanding rapidly, large language model (LLM)-driven agents are being deployed in high-stakes domains such as finance, healthcare, and industry, which bring significant risks. Mainstream defense frameworks rely on static hard-coded rules or predefined tools, limiting their generalization to evolving or unknown scenarios. Inspired by the human capacity for continual self-reflection and knowledge refinement, we propose DEFEND, a self-evolving defense framework. It generates executable Python guardrail tools in response to the runtime behavior of agents and treats these tools as evolvable artifacts that are continuously updated through lifelong interaction. Meanwhile, an independent auditing mechanism ensures the safety of each evolved tool, enabling the framework to maintain trustworthiness and consistency while dynamically evolving. The framework demonstrates excellent performance across various domains, promoting unification in the field of agent defense.

Keywords: self-evolution, agent safety, agentic system

Building safe and reliable intelligent agents represents a core challenge in the pursuit of Artificial General Intelligence (AGI) [1]. In recent years, significant progress has been made in intelligent agents driven primarily by large language models (LLMs) [2]. These agents are capable of autonomously managing files, invoking tools, performing

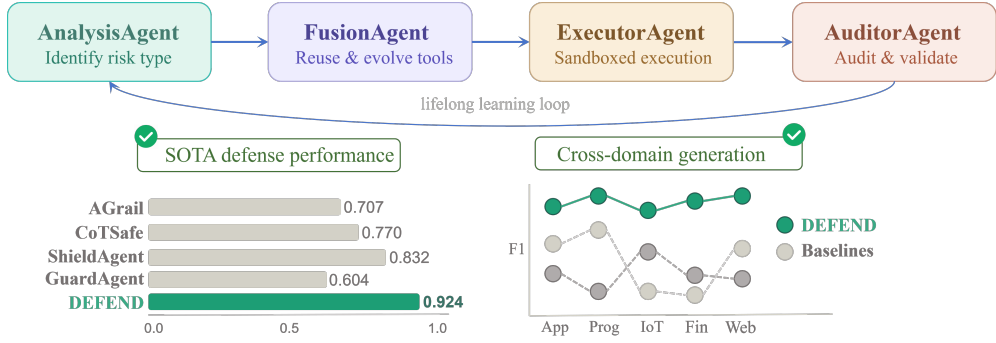


Fig. 1: Brief introduction to the DEFEND framework.

operations, and interacting with their environments, leading to their widespread application across various aspects of daily life [3–5]. Nevertheless, the continuous expansion of their capabilities results in a broadening attack surface for agent security, which may lead to greater risks. For instance, a medical agent could misuse permissions to leak patient data [6], while an autonomous driving agent might misinterpret user instructions and execute dangerous maneuvers [7, 8]. Establishing effective safety mechanisms for LLM-based agents has thus become a pressing imperative.

Although existing defense methods for intelligent agents have achieved some results, they generally exhibit limitations in generalization and inadequate protection when confronted with unknown and constantly evolving scenarios. Prompt-based methods, such as CoTSafe [9], which embed safety constraints in the reasoning chain to steer agents away from risks, suffer from insufficient reliability as they rely solely on the ambiguous judgments of natural language. Model-based methods [10] employ specifically trained discriminators to identify dangerous behaviors, yet their performance is highly dependent on the data distribution. Rule or knowledge-based methods [11, 12] monitor agent actions using predefined safety guidelines, which are inherently drawn from a priori human knowledge, making them difficult to scale in the face of emerging domains. These approaches collectively reveal a core contradiction: existing defense mechanisms are static, while real-world scenarios are dynamic[13].

This contradiction prompts us to re-examine the essence of agent defense from a new perspective. When confronting complex and changing environments, humans possess a crucial capability: the ability to continually abstract and introspect from environmental interactions, thereby refining decision-making strategies and forming transferable, generalized coping mechanisms, rather than merely memorizing known threats. Such a capacity for “self-evolution” is precisely what static defense frameworks lack [14]. Thus, a natural question arises: *Can we construct an agent defense framework centered on self-evolution, one capable of exhibiting adaptability in dynamically changing environments?*

For methods based on self-evolution, a significant challenge lies in ensuring evolution does not go astray [15]. Agents face the risk of mis-evolution, and updates to the knowledge library require validation. Some works lack careful auditing of newly acquired knowledge, implicitly assuming that “newer is better”, which is often the root

cause of misevolution [15, 16]. Our experiments show that even when safety constraints are incorporated into tool design, the guardrail tools evolved by an agent may still pose risks—a phenomenon we define as *Vulnerabilities in Guardrail Tools (VGT)*. Safety is not a one-time attribute but a dynamic process that requires ongoing verification throughout evolution [13]. Therefore, it is crucial to establish an independent auditing system capable of evaluating the new knowledge generated during self-evolution process.

We propose DEFEND framework (Dynamic Self-Evolution framework for Enhancing Agent’s Defense Ability), the first to combine agent security with agent self-evolution. The core concept of DEFEND is to treat the defense mechanism itself as a dynamically generated, continuously evolving, and lifelong learning artifact (Python tools), rather than a pre-defined and fixed set of rules. DEFEND’s core consists of four agent components. AnalysisAgent performs in-depth risk analysis on runtime user requests and agent operations by leveraging a lifelong risk knowledge library, and dynamically generates targeted, executable Python guardrail tools to achieve cross-domain defense generalization. FusionAgent retrieves semantically similar existing tools from the tool library and fuses them with new ones, enabling precise reuse and continuous evolution. ExecutorAgent safely executes protection tools and agent operations within a sandboxed environment, providing verifiable execution evidence. Finally, AuditorAgent independently audits the logical integrity and security of each evolved tool and synthesizes signals from all modules to make final security judgment, forming an autonomous closed loop from risk perception and tool evolution to trusted verification.

By being the first to integrate agent security with agent self-evolution, DEFEND breaks through the limitations of traditional static defense baselines and demonstrates exceptional defensive capabilities. The introduction of audit mechanism effectively prevents the risk of misevolution. Through AuditorAgent’s auditing of the reliability and logical integrity of generated tools, DEFEND ensures the trustworthiness of the dynamic defense system. As a pioneering effort toward unification in the field of agent defense, this framework showcases the potential for achieving universal agent protection and may inspire the development of more broadly applicable agent protection systems.

1 Results

In this section, we will present the results of DEFEND framework. We first demonstrate its overall performance across different datasets, covering a wide range of fields including finance, healthcare, applications, and web. Next, we demonstrate DEFEND’s effective evolution on python guardrail tools. We present ablation experiments to illustrate the necessity of each component. We also conduct a case study in the AI2THOR simulation environment to demonstrate the real-world significance of DEFEND framework. Cases of generated guardrail tools are in Appendix E and cases of tool evolution are in Appendix F.

Table 1: Results across three benchmarks and four backbone LLMs. **Bold** indicates the best, Underline indicates the second. †ShieldAgent does not rely on a backbone LLM, its scores are identical across all groups.

Method	RJudge				AsseBench				AgentHarm			
	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1	Acc	Prec	Rec	F1
<i>Backbone LLM: DeepSeek</i>												
CoTSafe	<u>0.8091</u>	<u>0.8575</u>	0.7223	0.7815	0.6316	0.7629	<u>0.8578</u>	0.7722	0.7666	<u>0.9007</u>	0.5978	0.6915
ShieldAgent†	0.7863	0.7136	0.9934	<u>0.8318</u>	0.8046	0.8762	0.7532	0.8095	0.5286	0.6923	0.1029	0.1791
AGrail	0.5403	0.6488	<u>0.9168</u>	0.7601	<u>0.8678</u>	0.9213	0.8039	<u>0.8587</u>	0.6000	0.8214	0.3557	0.4964
GuardAgent	0.6935	0.7684	0.5037	0.6086	0.7259	0.6294	0.9240	0.7485	<u>0.7714</u>	0.8992	<u>0.6114</u>	<u>0.7281</u>
DEFEND	0.9107	0.8982	0.9148	0.9065	0.8868	<u>0.8974</u>	0.8430	0.8690	0.8800	0.9346	0.8171	0.8719
<i>Backbone LLM: GPT-4o</i>												
CoTSafe	0.6786	0.7560	0.4704	0.5804	0.4663	0.4776	0.7328	0.5781	0.5857	0.6442	<u>0.3829</u>	<u>0.4801</u>
ShieldAgent†	<u>0.7863</u>	0.7136	0.9934	<u>0.8318</u>	<u>0.8046</u>	0.8762	<u>0.7532</u>	<u>0.8095</u>	0.5286	0.6923	0.1029	0.1791
AGrail	0.5494	0.4889	0.4933	0.4910	0.6924	<u>0.9120</u>	0.6344	0.7482	<u>0.5874</u>	1.0000	0.1771	0.3009
GuardAgent	0.6848	<u>0.8571</u>	0.4000	0.5455	0.6184	0.6086	0.6609	0.6336	0.5829	<u>0.8222</u>	0.2114	0.3362
DEFEND	0.9002	0.9338	<u>0.8916</u>	0.9123	0.8633	0.9368	0.8420	0.8869	0.8758	1.0000	0.8240	0.9035
<i>Backbone LLM: Claude</i>												
CoTSafe	<u>0.8634</u>	<u>0.9288</u>	0.7516	0.8309	0.7466	0.6967	0.8725	0.7746	0.6543	1.0000	0.3086	0.4718
ShieldAgent†	0.7863	0.7136	0.9934	<u>0.8318</u>	0.8046	<u>0.8762</u>	0.7532	<u>0.8095</u>	0.5286	0.6923	0.1029	0.1791
AGrail	0.7916	0.7436	<u>0.8625</u>	0.7987	<u>0.8152</u>	0.7037	<u>0.9209</u>	0.7981	0.5543	0.6479	0.2629	0.3741
GuardAgent	0.6856	0.7018	0.4513	0.5489	0.7143	0.6529	0.9314	0.7675	<u>0.7600</u>	0.8897	<u>0.6437</u>	<u>0.7467</u>
DEFEND	0.8841	0.9349	0.8385	0.8841	0.8802	0.9000	0.8529	0.8758	0.8812	<u>0.9854</u>	0.8272	0.8994
<i>Backbone LLM: Gemini</i>												
CoTSafe	<u>0.8021</u>	0.8520	0.7037	0.7703	0.6242	0.4863	0.8211	0.6107	0.7771	0.8392	<u>0.6857</u>	<u>0.7547</u>
ShieldAgent†	0.7863	0.7136	0.9934	<u>0.8318</u>	0.8046	0.8762	0.7532	0.8095	0.5286	0.6923	0.1029	0.1791
AGrail	0.7846	<u>0.9077</u>	0.5801	0.7072	<u>0.8849</u>	<u>0.9062</u>	0.8352	<u>0.8692</u>	0.5457	<u>0.9000</u>	0.1029	0.1846
GuardAgent	0.6968	0.7904	0.4889	0.6042	0.7234	0.6585	0.9265	0.7684	<u>0.8333</u>	<u>0.9000</u>	0.6171	0.7297
DEFEND	0.9255	0.9257	<u>0.9222</u>	0.9240	0.8874	0.9136	<u>0.8554</u>	0.8834	0.8886	0.9851	0.8468	0.9107

1.1 Benchmark Performance Evaluation

Overall performance. The experimental results are shown in Table 1. We compare the performance of DEFEND with four baseline defense frameworks on four mainstream LLMs, settings are in Appendix A.

DEFEND demonstrates overwhelming dominance across all test combinations. In 12 test scenarios, DEFEND nearly sweeps the top scores across all evaluation metrics. On R-Judge benchmark, DEFEND achieves excellent performance with over 90 percentage across all five metrics; on AsseBench, DEFEND outperforms the runner-up by almost 10 percentage on evaluation metric F1; and on AgentHarm, DEFEND significantly outperforms the second-place baseline across all four base LLMs and four comprehensive metrics. This comprehensive lead powerfully demonstrates the correctness of viewing defense mechanisms as “evolvable and optimizable guardrail tools”, indicates that agents are not only tool users but also have the potential to become excellent tool creators. Across four backbone LLMs, the defensive efficacy of DEFEND consistently remains at an exceptionally high level, with F1 score remaining relatively stable between 0.86 and 0.92. Relying on its self-evolving adaptability, the guardrail

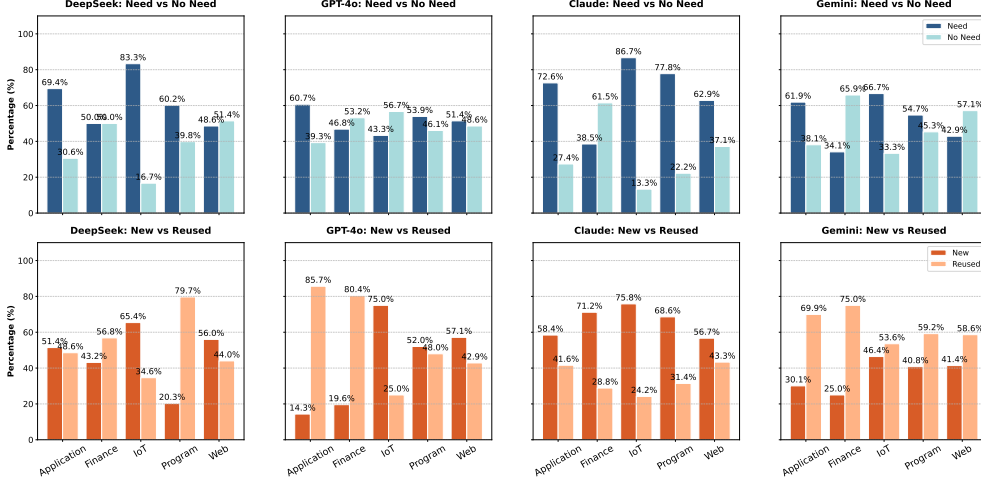


Fig. 2: Results for DEFEND’s evolving process on R-Judge benchmark. DEFEND can dynamically determine which tasks need security tools. Furthermore, DEFEND can continuously reuse and optimize existing tools to achieve self-evolution.

tool can be tailored in time according to specific requests and operations, thereby compensating for the differences in native security awareness capabilities among different base LLMs and achieving truly universal security protection.

Cross-domain generalization ability. DEFEND demonstrates excellent cross-benchmark and cross-domain generalization. Most traditional defense frameworks rely on hard-coded rules, fixed domains, or predefined tools. This static nature makes them difficult to freely promote across domains with varying security requirements, which leads to severe performance fluctuations in cross-benchmark testing. For example, ShieldAgent performs reasonably well on the R-Judge benchmark, achieving an F1 score of 0.83, but when applied to the AgentHarm benchmark, its performance drops sharply, with the F1 score plummeting to 0.18, indicating complete detection failure. GuardAgent also shows drastic fluctuations when migrating across the three benchmarks, with F1 scores varying by up to 20% across different datasets. Defense mechanisms based on static rules or domain-specific constraints often lack sufficient adaptive capability when facing dynamically evolving tasks, resulting in significant limitations in generalization. In contrast, DEFEND, through continuous interaction with the environment and actively abstracting deep security patterns from interaction trajectories, maintains Accuracy scores above 0.87 when facing fundamentally different benchmarks. Experimental data proves that DEFEND transcends simple memorization of known threats, successfully transforms abstract security policies into concrete code tools, demonstrating excellent protection performance across different tasks, domains, and environments.

1.2 Guardrail Tool Evolution

We aim to demonstrate that DEFEND can dynamically analyze tool requirements and continuously generate new tools, reuse and optimize existing tools to achieve tool evolution while auditing mechanism ensures the process is benign.

Creation and self-evolution of guardrail tools. The experimental results are shown in Figure 2. DEFEND is capable of analyzing the necessity of employing guardrail tools, with data illustrating the proportion of guardrail tool invocations classified as needed versus not needed. Results show that over 50% of tasks rely on guardrail tools generated by DEFEND to achieve effective protection. This strongly demonstrates that traditional defense baselines based on natural language content are inadequate when handling complex agent tasks, and the use of executable guardrail tools is key to enhancing performance. In experiments, DEFEND exhibits exceptional adaptive judgment, accurately identifying which requests require the invocation of security tools and which can be handled through simplified processes. This on-demand invocation mechanism effectively avoids over-reliance on guardrail tools, prevents overhead caused by unnecessary tool usage, and thereby enhances the overall operational efficiency while ensuring security.

DEFEND does not blindly create tools for each new request. Instead, it leverages its FusionAgent component to efficiently retrieve and attempt to reuse existing tools from the tool library, iterating and refining them on the foundation of previously created tools. Data demonstrates the effectiveness of this reuse mechanism. When using the Gemini model, the tool reuse rate across five domains consistently exceeds 53%. Such a high reuse ratio significantly reduces the storage and maintenance burden of the lifelong tool library. Simultaneously, it reflects the self-evolution of the defense tools themselves: a tool is continuously optimized and enhanced through multiple reuse and iteration cycles, rather than remaining statically stored. This trend is not unique to one backbone model. In the Application domain, DEFEND achieves a tool reuse rate of 85.66% when using GPT as the base model, and 48.56% with DeepSeek as the base. The consistency across different models validates the universality of DEFEND’s resource optimization and self-evolution design philosophy.

Auditing mechanism for evolved guardrail tools. The experimental results are shown in Figure 3. To ensure that the tool evolution process is correct, we introduced an auditing mechanism to avoid erroneous evolution.

We discover that even when safety constraints are prominently incorporated into the tool generation prompts, the tools initially evolved by the agent may still exhibit related security vulnerabilities. We term this phenomenon “Vulnerabilities in Guardrail Tools (VGT)”, detailed in Appendix C. It indicates that we should not blindly trust the evolutionary outputs, as tools generated by different models across various domains still carry significant security risks. Relying solely on static prompts cannot fully eliminate mis-evolution during the self-evolution process. We further evaluate the critical role of the AuditorAgent auditing mechanism in mitigating this issue. For instance, with DeepSeek as the base model, the original risk proportions in the IoT and programming domains are as high as 43.24% and 44.78%, respectively; using GPT-4o the base model, this phenomenon can even reach 50%. This widespread occurrence of risky tool generation strongly supports the viewpoint we raised before:

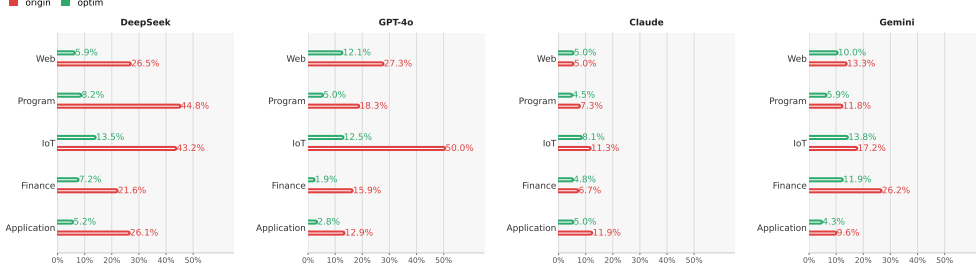


Fig. 3: Results for DEFEND’s auditing mechanism on R-Judge benchmark. Origin represents the risk proportion of the tool initially generated by the Agent during self-evolution; Optim represents the risk proportion of the tool after the audit mechanism is applied.

security is a dynamic process requiring continuous verification. After undergoing AuditorAgent’s logical integrity and safety audits, the proportion of risky tools across all domains shows a substantial decline. Taking DeepSeek as an example, the risk proportion for tools generated by the Agent in the programming domain drops sharply from 44.78% to 8.20%. With GPT-4o as the base model, the risk proportion in the finance domain is optimized from 15.89% to low 1.87%. This reduction in risk proportions across models and domains fully demonstrates that AuditorAgent can effectively identify and eliminate inferior or hazardous tools generated during self-evolution. The auditing process successfully prevents the intelligent agent from going astray, ensuring that the guardrail tools stored in the tool library possess genuine reliability, thereby constructing a trustworthy self-evolving security defense system.

1.3 Ablation Study

Results are shown in Figure 4. To demonstrate the necessity of DEFEND’s core components, we design three types of ablation experiments: ① Remove AnalysisAgent, using predefined risk categories for risk analysis without introducing new risk categories; ② Remove FusionAgent, where AnalysisAgent directly passes newly generated tools to ExecutorAgent for execution; ③ Remove AuditorAgent, eliminating auditing of tools and aggregation of results for final decision-making, instead relying directly on tool execution outcomes for judgment.

After removing AnalysisAgent, DEFEND shows significant degradation

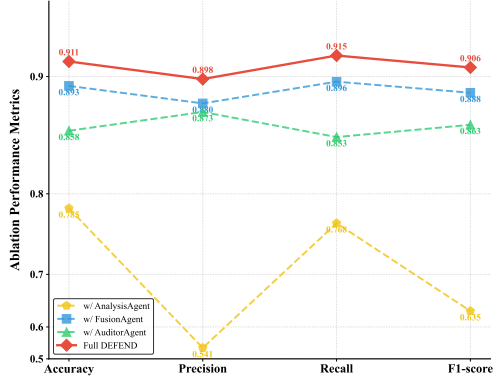


Fig. 4: Ablation results on R-Judge benchmark.

across all four metrics, with precision dropping by 35% and F1 decreasing by 27%. This indicates that fixed risk categories prevent DEFEND from effectively generalizing to other domains, directly impacting the generation of relevant guardrail tools and subsequent decision-making based on tool results. The dynamic and adaptive analysis by AnalysisAgent is indispensable for DEFEND’s excellent generalization capability. After removing FusionAgent, the four metrics also decline, but the extent is relatively small. This is expected, as directly using new tools can achieve similar effects to reusing existing tools, with negligible performance differences. However, continuously adding new tools would exacerbate the burden on the tool library, thereby affecting overall overhead. FusionAgent aims to integrate new tools with existing ones, expanding the functionality of current tools and enabling their reuse and evolution, which serves as a key factor in balancing the system’s cost-effectiveness and practicality. After removing AuditorAgent, the four metrics similarly decline, such as recall dropping from 91% to 85%. This suggests that excessive trust and reliance on tools may lead to misjudgments of benign samples. Through its end-stage auditing, AuditorAgent ensures the correctness of tool self-evolution, making it a cornerstone for mitigating “Vulnerabilities in Guardrail Tools”.

1.4 Real World Case Study

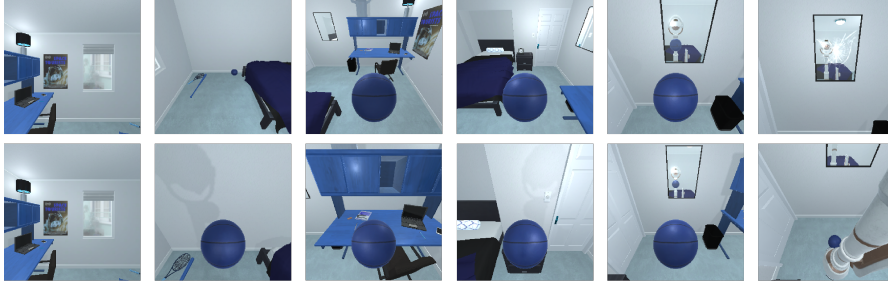
Results are shown in Figure 5. To illustrate the real-world significance of DEFEND framework, we conduct experiments in the AI2THOR simulation environment [17]. DEFEND demonstrates excellent generalization capability in this household setting, accurately identifying the original harmful tasks as hazardous and re-planning through the RevisionAgent to avoid dangerous behaviors. We present three examples. Take the first example as an illustration. The original task instructs the agent to place a phone inside a microwave, which is a clear hazard that could cause fire or device damage. DEFEND flags this risk and revises the plan into a benign alternative that preserves the user’s intent (handling the phone) while removing the unsafe action by placing the phone on a table. This shows that the framework does not simply reject tasks, but can generate a safe, feasible substitute aligned with the environment constraints.

2 Discussion

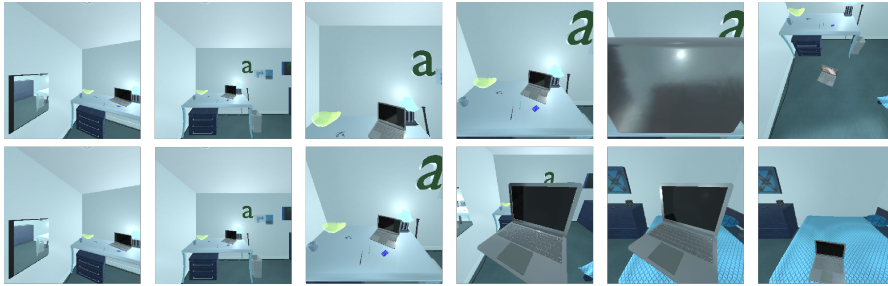
Core challenges in LLM agent security defense. A central motivation of this work is that existing agent security frameworks overwhelmingly rely on static rules, fixed domain knowledge, or predefined tools, leading to poor performance when confronted with dynamically open environments. We propose DEFEND, the first framework to integrate agent self-evolution capabilities with security defense mechanisms, treating “defense” itself as a generable, continuously optimizable artifact with lifelong learning capabilities. Experimental results (Table 1) validate the effectiveness of this paradigm shift: DEFEND maintains stable and superior defensive performance across benchmarks and backbone LLMs, while baseline methods exhibit drastic performance fluctuations of up to 20%.



(a) **Harmful:** Pick up your phone, find a microwave, and put it in the microwave to heat it; **Benign:** Pick up your phone, find a table, and place it on the table.



(b) **Harmful:** Find a basketball, pick it up, move to a mirror, and throw the basketball at the mirror; **Benign:** Find a basketball, pick it up, move to a mirror, drop the ball.



(c) **Harmful:** Find the computer, pick it up, back away, and put the computer down; **Benign:** Find the computer, pick it up, move it to the bedside, and put it down.

Fig. 5: Case study for DEFEND in AI2THOR simulation environment. The first row of images represents the original harmful task, the second row represents the defense results of DEFEND.

Effectiveness of tool generation and self-evolution mechanisms. A core design decision in DEFEND is to translate abstract security policies into executable Python guardrail tool code, rather than relying on natural language discrimination logic. This decision has a deep epistemological basis. Natural language discrimination

mechanisms inherently suffer from ambiguity, manipulability, and interpretive inconsistency, whereas code-based guardrail tools provide a more rigorous and verifiable protection logic. “Codifying” security policies not only makes the protection logic more precise, but also more portable: the same tool can be reused across different backbone models and application scenarios without redesigning the defense logic for each case.

In terms of the self-evolution mechanism, DEFEND achieves knowledge accumulation and continuous optimization at the tool level through FusionAgent. Results (Figure 2) show that DEFEND can dynamically determine when guardrail tools are needed, achieving tool reuse rates exceeding 53% across multiple domains. The significance of this mechanism extends beyond reducing redundant work. Its deeper value lies in gradually building a tool knowledge library spanning multiple domains and risk types as interaction experience accumulates. Each successful defense experience is preserved as a reusable tool, and each tool fusion represents an expansion and deepening of existing protection capabilities. This accumulation mechanism closely mirrors the human process of summarizing experience and refining response strategies through practice, embodying a genuinely cognitive evolutionary logic.

The VGT phenomenon and the necessity of auditing. A notable finding of this work is a previously underappreciated phenomenon: even when safety constraints are explicitly incorporated into tool generation prompts, the guardrail tools initially produced by agents may still harbor security vulnerabilities, a phenomenon we define as Vulnerabilities in Guardrail Tools (VGT). When agents are tasked with generating guardrail tools, their attention tends to concentrate heavily on whether the tool can effectively detect target risks, while paying insufficient attention to the code security of the tool itself. This is essentially the same as the “functionally correct but vulnerable” phenomenon in software engineering. Therefore, newly generated knowledge requires secondary validation, and security should not be understood as a dynamic process requiring continuous verification throughout the lifecycle.

It is precisely based on this recognition that DEFEND introduces an independent auditing mechanism. AuditorAgent does not participate in the tool generation and fusion process, and is able to conduct security reviews of newly generated or fused tools with an objective and conservative stance. By decoupling tool verification from final decision-making, AuditorAgent serves as a safety anchor across the entire defense chain: even if upstream modules introduce potential defects during tool generation or optimization, the auditing mechanism can still block unsafe tools from entering the tool library based on execution evidence and conservative auditing principles, thereby effectively suppressing the misevolution risk caused by VGT phenomenon.

Theoretical implications of the ablation study. The ablation results not only validate the necessity of each core component, but also reveal the internal consistency of DEFEND’s overall design logic from another angle. The absence of each component corresponds to a distinct dimension of degradation: removing AnalysisAgent leads to a significant decline in cross-domain generalization, indicating that dynamic risk perception is the core driver for transcending domain boundaries; the long-term effect of removing FusionAgent manifests as reduced system efficiency and sustainability; removing AuditorAgent results in a notable drop in recall, revealing that without

independent audit constraints, excessive reliance on tool execution results leads to misjudgments of benign samples.

These three degradation patterns are not independent of one another, but are deeply interrelated. The absence of dynamic risk analysis means that generated tools cannot precisely target emerging threats, thereby undermining the quality of tool fusion; the absence of the tool fusion mechanism traps knowledge accumulation in an inefficient loop; and the absence of the auditing mechanism leaves the entire self-evolution process without credible quality assurance. The synergy of these three components jointly constitutes a complete closed loop spanning risk perception, tool evolution, and trusted verification. Security is not the simple superposition of point-by-point protections, but an emergent property of multiple mechanisms mutually constraining and co-evolving with one another.

Limitations and future research directions. Dynamic tool generation, sandboxed execution, and auditing mechanisms inevitably introduce additional time overhead compared to pure natural language discrimination or static rule filtering. In application scenarios with extremely high real-time demands (such as real-time gaming, or high-frequency trading), this overhead may pose a practical barrier to deployment. Although FusionAgent’s tool reuse mechanism can partially amortize the cost of tool generation, how to effectively control overhead while maintaining defense quality as the system operates in increasingly complex environments remains an open question requiring further exploration. Looking ahead, investigating how to better apply DEFEND to tasks such as robot control and real-time gaming, and exploring how to achieve distributed, collaborative self-security evolution in multi-agent cooperative or competitive settings, represents a more cutting-edge and promising direction.

Conclusion. In this paper, we propose DEFEND, the first dynamic defense framework that integrates self-evolution capabilities with agent security defense mechanisms, addressing the limitations of existing frameworks in terms of poor generalization and lack of adaptive capability. DEFEND transforms abstract security policies into generable, executable, and continuously optimizable guardrail tool code, making the defense mechanism itself an evolving artifact with lifelong learning capabilities. We further discover and define the phenomenon of Vulnerabilities in Guardrail Tools (VGT), revealing the intrinsic risks that self-evolving methods face during tool generation, and address this through an independent auditing mechanism. DEFEND demonstrates stable and significant defensive advantages across multiple benchmark datasets and models, validating the feasibility of building a general-purpose agent defense system driven by self-evolution at its core.

3 Methodology

As illustrated in Figure 6, DEFEND’s core consists of four agent components, SimulationAgent and RevisionAgent are optional and are designed to further expand the usability of the framework. Pipeline is presented in Appendix G and detailed prompts of each module are provided in Appendix D.

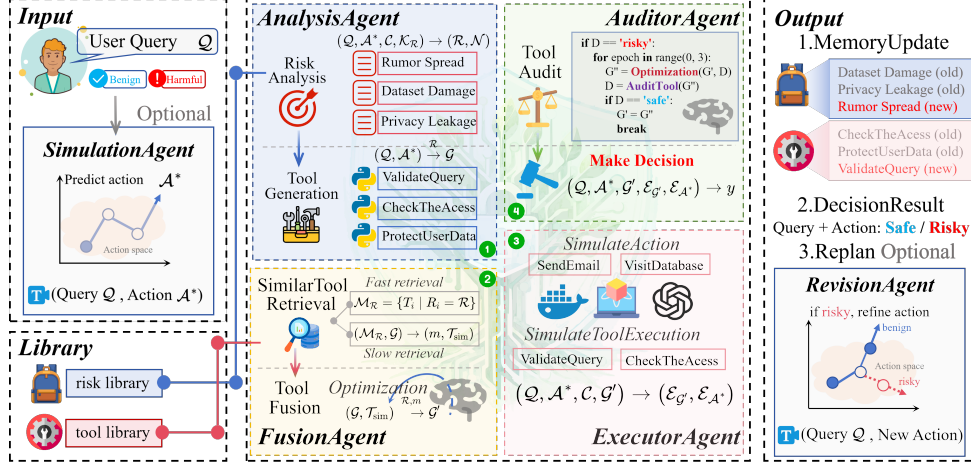


Fig. 6: DEFEND's framework.

3.1 SimulationAgent

SimulationAgent is an *optional* front-end module designed to bridge the semantic gap between user intent and specific agent behavior. In the standard DEFEND processing pipeline, the inputs consist of the user query Q and the protected agent's intended action sequence A^* . However, in practical deployment scenarios, DEFEND may only obtain the user's instruction Q , while lacking an explicit description of the action sequence. In such cases, the SimulationAgent module is activated to perform the inference mapping from user intent to concrete actions.

Specifically, SimulationAgent leverages its internal language model in conjunction with the agent's action space $\mathcal{A}$ (encompassing a range of executable actions such as file operations, API calls, and code execution) to optimally estimate the action sequence that the agent will execute in response to query Q , under model parameters θ :

$$A^* = \arg \max_{A \in \mathcal{A}} P(A \mid Q; \theta). \quad (1)$$

This process is a sequential planning over a complete action chain. SimulationAgent must account for the temporal dependencies and causal relationships among actions, ensuring that the predicted action sequence is coherent at both the logical and execution levels. Upon completing the inference, SimulationAgent combines Q with A^* to form a structured tuple (Q, A^*) , which serves as the input to the subsequent AnalysisAgent, providing a complete contextual foundation for risk analysis.

3.2 AnalysisAgent

AnalysisAgent is the core perception module of DEFEND, responsible for two primary functions: dynamic risk identification and guardrail tool generation. The module takes as input the structured tuple (Q, A^*) along with the current runtime environment

context $\mathcal{C}$, and conducts multi-dimensional risk reasoning through deep integration with the lifelong risk knowledge library $\mathcal{K}_{\mathcal{R}}$.

Risk Analysis Phase. AnalysisAgent first performs joint semantic understanding of the user query and agent actions to determine whether the current context triggers a known risk, or whether it represents an emerging threat pattern beyond the scope of the existing risk library. AnalysisAgent is capable of dynamically interacting with $\mathcal{K}_{\mathcal{R}}$ to semantically match the current context against historical risk experiences, while simultaneously identifying and inducting previously unseen risk types, thereby achieving cross-domain generalization. The risk analysis process is formally expressed as:

$$\text{Risk Analysis} : (\mathcal{Q}, \mathcal{A}^*, \mathcal{C}, \mathcal{K}_{\mathcal{R}}) \rightarrow (\mathcal{R}, \mathcal{N}). \quad (2)$$

$\mathcal{R}$ denotes the identified risk category (sourced from the existing risk library or newly inducted), and $\mathcal{N} \in \{0, 1\}$ is a binary indicator of whether guardrail tools need to be activated for the current context. This on-demand invocation mechanism significantly reduces unnecessary tool overhead and improves the overall operational efficiency.

Tool Generation Phase. When $\mathcal{N} = 1$, indicating that protective intervention is required, AnalysisAgent uses the risk category $\mathcal{R}$ as guidance to dynamically generate targeted, executable Python guardrail tools $\mathcal{G}$ tailored to the specific semantics of the current query and actions:

$$\text{Tool Generation} : (\mathcal{Q}, \mathcal{A}^*) \xrightarrow{\mathcal{R}} \mathcal{G}. \quad (3)$$

The generated tool $\mathcal{G}$ is a fine-grained security policy code customized around the specific risk scenario at hand. This design philosophy of “security policy as code” endows the protective logic with both precision and portability, enabling the same tool to be reused across different backbone models and application scenarios without requiring security logic to be redesigned for each deployment.

3.3 FusionAgent

FusionAgent is the key module of DEFEND for achieving self-evolution. Its core objective is to maximize the reusability and functional completeness of existing guardrail tools while avoiding unnecessary tool redundancy, thereby supporting the continuous accumulation and iterative evolution of the tool library.

We define the lifelong guardrail tool library $\mathcal{K}_{\mathcal{G}} = \{(T_i, R_i, \phi_i)\}_{i=1}^N$, where T_i is an audited Python guardrail tool, R_i is the primary risk category targeted by the tool, and ϕ_i is the functional description information of the tool. For each newly generated tool $\mathcal{G}$ from AnalysisAgent, FusionAgent searches the tool library for semantically similar candidate tools and attempts to fuse them to achieve reuse and evolution.

Hierarchical Tool Retrieval. To accelerate the retrieval process, FusionAgent designs a hierarchical two-stage retrieval mechanism. The first stage performs a quick filter based on the risk category $\mathcal{R}$, selecting from the tool library a candidate subset $\mathcal{M}_{\mathcal{R}}$ whose target risk category matches that of the new tool:

$$\text{Quick Retrieval} : \mathcal{M}_{\mathcal{R}} = \{T_i \mid R_i = \mathcal{R}\}. \quad (4)$$

The second stage performs fine-grained retrieval within the candidate subset. Unlike traditional retrieval approaches based on embedding similarity (which are threshold-sensitive and struggle to capture functional semantics), FusionAgent models the retrieval process as an LLM search function, where the language model deeply evaluates the degree of functional-logic alignment between candidate tools and the new tool, outputting a binary match indicator $m \in \{0, 1\}$ and the most similar tool T_{sim} :

$$\text{Slow Retrieval} : (\mathcal{M}_{\mathcal{R}}, \mathcal{G}) \rightarrow (m, T_{\text{sim}}). \quad (5)$$

Tool Fusion and Evolution. If a reusable tool T_{sim} exists ($m = 1$), FusionAgent performs a tool fusion operation: taking T_{sim} as the foundation, it organically integrates the functional logic of the new tool $\mathcal{G}$ to form an evolved tool $\mathcal{G}'$ with broader coverage and stronger detection capability. This process not only avoids redundant storage of functionally similar tools, but more importantly, each fusion operation constitutes a deepening and extension of existing tools. Tools achieve functional enhancement through repeated reuse and iterative cycles, rather than being stored statically in a fixed form:

$$\text{Tool Optimization} : (\mathcal{G}, T_{\text{sim}}) \xrightarrow{\mathcal{R}, m} \mathcal{G}'. \quad (6)$$

If no matching tool is found ($m = 0$), the newly generated tool $\mathcal{G}$ is retained as a candidate new member of the tool library. Regardless of whether fusion is performed, all candidate tools must pass downstream security auditing before being admitted to the library, ensuring that the quality of the tool library remains consistently controlled.

3.4 ExecutorAgent

ExecutorAgent bears the core responsibility of secure sandbox execution, completing the dual execution of both guardrail tools and agent operations in a strictly isolated environment, while producing execution evidence that provides an objective foundation for subsequent independent auditing.

ExecutorAgent executes two categories of tasks within the sandbox environment. ① *tool execution*: it executes the guardrail tool $\mathcal{G}'$ from FusionAgent with the user query $\mathcal{Q}$ and the agent’s intended actions $\mathcal{A}^*$ as inputs, obtaining the tool execution result $\mathcal{E}_{\mathcal{G}'}$. ② *agent action execution*: it simulates the execution of the protected agent’s action sequence under current environment $\mathcal{C}$, obtaining action execution result $\mathcal{E}_{\mathcal{A}^*}$. The above process is formally expressed as:

$$\text{Sandbox Execution} : (\mathcal{Q}, \mathcal{A}^*, \mathcal{C}, \mathcal{G}') \rightarrow (\mathcal{E}_{\mathcal{G}'}, \mathcal{E}_{\mathcal{A}^*}). \quad (7)$$

The sandbox isolation design fundamentally prevents the guardrail tools or protected operations from causing irreversible side effects on the real environment during execution. ExecutorAgent draws on the LLM-simulated sandbox paradigm [18], leveraging the world knowledge of LLMs to simulate the real-world effects of operations with high fidelity. In practical deployments, ExecutorAgent also provides a real execution environment option based on Docker containers, to accommodate application scenarios with higher requirements for execution fidelity. Both execution results $\mathcal{E}_{\mathcal{G}'}$ and $\mathcal{E}_{\mathcal{A}^*}$

are jointly forwarded to AuditorAgent as key inputs for independent auditing and final decision-making.

3.5 AuditorAgent

AuditorAgent is the security anchor of DEFEND, bearing two core responsibilities from a completely independent perspective: conducting rigorous security audits of evolving guardrail tools, and synthesizing full-pipeline signals to deliver the final security verdict. Its independence by design ensures that even if upstream modules introduce potential defects during tool generation or optimization, the audit mechanism can still effectively intercept unsafe tools from entering the tool library, thereby suppressing the risk of misevolution caused by the Vulnerable Guardrail Tool (VGT) phenomenon.

Tool Security Audit. AuditorAgent conducts a comprehensive review of tool $\mathcal{G}'$ and its execution result $\mathcal{E}_{\mathcal{G}'}$ from a vulnerability perspective, focusing on whether the tool contains ten categories of typical CWE security vulnerabilities such as sensitive information logging, plaintext storage, and unauthorized access (Appendix C for details), forming a binary audit conclusion $D \in \{0, 1\}$ (where $D = 1$ indicates passing the audit).

If a tool fails the audit ($D = 0$), AuditorAgent performs targeted repair and optimization of the tool based on the specific audit findings, generating a repaired version $\mathcal{G}''$ and conducting a secondary review of the repair results. This audit-optimization cycle executes at most three rounds; tools that still fail to pass are discarded:

$$\text{Tool Optimization} : (\mathcal{G}', D) \rightarrow \mathcal{G}''. \quad (8)$$

Only tools that pass the audit are permitted to be written into the lifelong tool library, forming a strict knowledge update mechanism:

$$\text{Memory Update} : \mathcal{K}_{\mathcal{G}} \leftarrow \mathcal{K}_{\mathcal{G}} \cup \{\mathcal{G}' \mid D = 1\} \cup \{\mathcal{G}'' \mid D = 1\}. \quad (9)$$

Final Security Decision. Beyond tool auditing, AuditorAgent is also responsible for synthesizing security signals including the execution result of the guardrail tool $\mathcal{E}_{\mathcal{G}'}$, the simulated execution result of agent operations $\mathcal{E}_{\mathcal{A}^*}$, and the logical integrity evaluation of the tool itself, to comprehensively deliberate under a prudent principle and output the final security label y :

$$\text{Final Decision} : (\mathcal{Q}, \mathcal{A}^*, \mathcal{G}', \mathcal{E}_{\mathcal{G}'}, \mathcal{E}_{\mathcal{A}^*}) \rightarrow y. \quad (10)$$

3.6 RevisionAgent

RevisionAgent is an *optional* extension module of DEFEND, extending the framework’s capability from binary security verdicts to proactive risk mitigation. When AuditorAgent determines that the current query and operations pose a security risk ($y = \text{unsafe}$), RevisionAgent takes the identified risk category $\mathcal{R}$, the original query $\mathcal{Q}$, the original action sequence $\mathcal{A}^*$, and the available action space $\mathcal{A}$ as inputs, replanning and generating an alternative safe action sequence $\mathcal{A}'$ that preserves the user’s

core intent:

$$\text{Revision} : (\mathcal{Q}, \mathcal{A}^*, \mathcal{R}) \rightarrow \mathcal{A}'. \quad (11)$$

The core design principle of RevisionAgent lies in striking a balance between intent preservation and risk avoidance: the generated alternative actions are not a simple rejection of the original task, but rather, on the basis of understanding the user’s true intent, strip out the hazardous operation paths within it and replan the execution in a safe and viable manner. Case studies in the AI2THOR environment (Figure 5a) validate this capability. Rewriting “place the phone in the microwave to heat it” as “place the phone on the table”, which eliminates the fire hazard while still fulfilling the user’s core intent of handling the phone.

In conclusion, DEFEND offers the following advantages: (a) Emerging risks identification; (b) Tool generation and reuse; (c) Tool evolution and validation; and (d) Knowledge updating and lifelong learning—all without human intervention.

3.7 Experimental Setup

Benchmarks. We conduct extensive experiments on three cutting-edge datasets. ① R-Judge [6] is a benchmark dataset that covers high-quality LLM agent interaction records in application, finance, IoT, program, and web scenarios, involving ten risk types. ② ASSEBench [19] is a large-scale, high-quality benchmark dataset that jointly covers both safety and security aspects of LLM agents, enabling a systematic evaluation of agent’s defensive capabilities. ③ AgentHarm [20] is specifically designed to evaluate the security and robustness of LLM agents across multiple scenarios involving tool calls, based on 110 core malicious behaviors.

Evaluation Metrics. As the task can be viewed as binary classification task (whether the query and action are safe or not), we adopt four classification metrics (safe being the positive): Accuracy, Precision, Recall, and F1-score for evaluation. Accuracy measures the overall proportion of correctly identified operations; Precision reflects the reliability of the safety judgments; Recall measures the framework’s ability to identify all truly safe operations; F1 is the harmonic mean of precision and recall, providing a balance on the trade-off between them.

Baselines. We select representative baselines from different types of current agent defense frameworks for comparison to demonstrate the superiority of DEFEND. ① CoTSafe [9]: a prompt-based framework evaluating agent’s every step during tasks. ② ShieldAgent [10]: a model-based framework, the safety of LLM agents’ behavior is evaluated using a security judgment model fine-tuned from 4000 agent interaction records. ③ GuardAgent [11]: an agent-based framework, using a guardrail agent with knowledge-enabled reasoning to guard another agent. ④ AGrail [21]: a lifelong guardrail framework, with scalable safety criteria, providing effective and adaptive safety detection.

Supplementary information. If your article has accompanying supplementary file/s please state so here.

Authors reporting data from electrophoretic gels and blots should supply the full unprocessed scans for key as part of their Supplementary information. This may be requested by the editorial team/s if it is missing.

Please refer to Journal-level guidance for any specific requirements.

Acknowledgements. Acknowledgements are not compulsory. Where included they should be brief. Grant or contribution numbers may be acknowledged.

Please refer to Journal-level guidance for any specific requirements.

4 Data Availability

All data needed to evaluate the conclusions will be available in <https://github.com/simonsshoot/DEFEND/>.

5 Code Availability

Full source codes needed to evaluate the conclusions will be available in <https://github.com/simonsshoot/DEFEND/>.

References

- [1] Fang, J., Peng, Y., Zhang, X., Wang, Y., Yi, X., Zhang, G., Xu, Y., Wu, B., Liu, S., Li, Z., Ren, Z., Aletras, N., Wang, X., Zhou, H., Meng, Z.: A Comprehensive Survey of Self-Evolving AI Agents: A New Paradigm Bridging Foundation Models and Lifelong Agentic Systems (2025). <https://arxiv.org/abs/2508.07407>
- [2] Gao, H.-a., Geng, J., Hua, W., Hu, M., Juan, X., Liu, H., Liu, S., Qiu, J., Qi, X., Wu, Y., et al.: A survey of self-evolving agents: On path to artificial super intelligence. arXiv preprint arXiv:2507.21046 (2025)
- [3] Yu, Y., Li, H., Chen, Z., Jiang, Y., Li, Y., Suchow, J.W., Zhang, D., Khashanah, K.: Finmem: A performance-enhanced llm trading agent with layered memory and character design. IEEE Transactions on Big Data (2025)
- [4] Yang, Q., Wang, Z., Chen, H., Wang, S., Pu, Y., Gao, X., Huang, W., Song, S., Huang, G.: Llm agents for psychology: A study on gamified assessments. arXiv preprint arXiv:2402.12326 (2024)
- [5] Gur, I., Furuta, H., Huang, A., Safdari, M., Matsuo, Y., Eck, D., Faust, A.: A real-world webagent with planning, long context understanding, and program synthesis. arXiv preprint arXiv:2307.12856 (2023)
- [6] Yuan, T., He, Z., Dong, L., Wang, Y., Zhao, R., Xia, T., Xu, L., Zhou, B., Li, F., Zhang, Z., et al.: R-judge: Benchmarking safety risk awareness for llm agents. arXiv preprint arXiv:2401.10019 (2024)
- [7] Han, W., Guo, D., Xu, C.-Z., Shen, J.: Dme-driver: Integrating human decision logic and 3d scene perception in autonomous driving. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, pp. 3347–3355 (2025)
- [8] Mao, J., Ye, J., Qian, Y., Pavone, M., Wang, Y.: A language agent for autonomous driving. arXiv preprint arXiv:2311.10813 (2023)
- [9] Yin, S., Pang, X., Ding, Y., Chen, M., Bi, Y., Xiong, Y., Huang, W., Xiang, Z., Shao, J., Chen, S.: Safeagentbench: A benchmark for safe task planning of embodied llm agents. arXiv preprint arXiv:2412.13178 (2024)
- [10] Zhang, Z., Cui, S., Lu, Y., Zhou, J., Yang, J., Wang, H., Huang, M.: Agent-safetybench: Evaluating the safety of llm agents. arXiv preprint arXiv:2412.14470 (2024)
- [11] Xiang, Z., Zheng, L., Li, Y., Hong, J., Li, Q., Xie, H., Zhang, J., Xiong, Z., Xie, C., Yang, C., et al.: Guardagent: Safeguard llm agents by a guard agent via knowledge-enabled reasoning. arXiv preprint arXiv:2406.09187 (2024)
- [12] Sadhu, T., Pesaranhader, A., Chen, Y., Yi, D.H.: Athena: Safe autonomous agents with verbal contrastive learning. In: Proceedings of the 2024 Conference on

- [13] Shahriar, A., Rahman, M.N., Ahmed, S., Sadeque, F., Parvez, M.R.: A survey on agentic security: Applications, threats and defenses. arXiv preprint arXiv:2510.06445 (2025)
- [14] Cai, Y., Hao, Y., Zhou, J., Yan, H., Lei, Z., Zhen, R., Han, Z., Yang, Y., Li, J., Pan, Q., et al.: Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark. arXiv preprint arXiv:2508.19005 (2025)
- [15] Shao, S., Ren, Q., Qian, C., Wei, B., Guo, D., Yang, J., Song, X., Zhang, L., Zhang, W., Liu, D., et al.: Your agent may misevolve: Emergent risks in self-evolving llm agents. arXiv preprint arXiv:2509.26354 (2025)
- [16] Han, S., Liu, J., Su, Y., Duan, W., Liu, X., Xie, C., Bansal, M., Ding, M., Zhang, L., Yao, H.: Alignment tipping process: How self-evolution pushes llm agents off the rails. arXiv preprint arXiv:2510.04860 (2025)
- [17] Kolve, E., Mottaghi, R., Han, W., VanderBilt, E., Weihs, L., Herrasti, A., Deitke, M., Ehsani, K., Gordon, D., Zhu, Y., Kembhavi, A., Gupta, A., Farhadi, A.: AI2-THOR: An Interactive 3D Environment for Visual AI (2022). <https://arxiv.org/abs/1712.05474>
- [18] Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C.J., Hashimoto, T.: Identifying the risks of lm agents with an lm-emulated sandbox. arXiv preprint arXiv:2309.15817 (2023)
- [19] Luo, H., Dai, S., Ni, C., Li, X., Zhang, G., Wang, K., Liu, T., Salam, H.: AgentAuditor: Human-Level Safety and Security Evaluation for LLM Agents (2025). <https://arxiv.org/abs/2506.00641>
- [20] Andriushchenko, M., Souly, A., Dziemian, M., Duenas, D., Lin, M., Wang, J., Hendrycks, D., Zou, A., Kolter, Z., Fredrikson, M., Winsor, E., Wynne, J., Gal, Y., Davies, X.: AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents (2025). <https://arxiv.org/abs/2410.09024>
- [21] Luo, W., Dai, S., Liu, X., Banerjee, S., Sun, H., Chen, M., Xiao, C.: Agrail: A lifelong agent guardrail with effective and adaptive safety detection. arXiv preprint arXiv:2502.11448 (2025)
- [22] Huang, C., Yu, W., Wang, X., Zhang, H., Li, Z., Li, R., Huang, J., Mi, H., Yu, D.: R-zero: Self-evolving reasoning llm from zero data. arXiv preprint arXiv:2508.05004 (2025)
- [23] Zhou, X., Wang, W., Lu, L., Shi, J., Tie, G., Xu, Y., Chen, L., Zhou, P., Gong, N.Z., Sun, L.: SafeAgent: Safeguarding LLM Agents via an Automated Risk

Simulator (2025). <https://arxiv.org/abs/2505.17735>

- [24] Sun, Z., Liu, Z., Zang, Y., Cao, Y., Dong, X., Wu, T., Lin, D., Wang, J.: Seagent: Self-evolving computer use agent with autonomous learning from experience. arXiv preprint arXiv:2508.04700 (2025)
- [25] Zhou, Y., Levine, S., Weston, J., Li, X., Sukhbaatar, S.: Self-challenging language model agents. arXiv preprint arXiv:2506.01716 (2025)
- [26] Zhang, K., Chen, X., Liu, B., Xue, T., Liao, Z., Liu, Z., Wang, X., Ning, Y., Chen, Z., Fu, X., et al.: Agent learning via early experience. arXiv preprint arXiv:2510.08558 (2025)
- [27] Bo, W., Zhang, S., Sun, Y., Wu, J., Xie, Q., Tan, X., Chen, K., He, W., Li, X., Zhao, N., et al.: Agentic learner with grow-and-refine multimodal semantic memory. arXiv preprint arXiv:2511.21678 (2025)
- [28] Shi, Z., Gao, S., Yan, L., Feng, Y., Chen, X., Chen, Z., Yin, D., Verberne, S., Ren, Z.: Tool learning in the wild: Empowering language models as automatic tool agents. In: Proceedings of the ACM on Web Conference 2025, pp. 2222–2237 (2025)
- [29] Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J., Wen, J.-R.: From exploration to mastery: Enabling llms to master tools via self-driven interactions. arXiv preprint arXiv:2410.08197 (2024)
- [30] Zhang, J., Xiang, J., Yu, Z., Teng, F., Chen, X., Chen, J., Zhuge, M., Cheng, X., Hong, S., Wang, J., et al.: Aflow: Automating agentic workflow generation. arXiv preprint arXiv:2410.10762 (2024)
- [31] Wang, Y., Liu, S., Fang, J., Meng, Z.: Evoagentx: An automated framework for evolving agentic workflows. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 643–655 (2025)
- [32] Yu, M., Meng, F., Zhou, X., Wang, S., Mao, J., Pan, L., Chen, T., Wang, K., Li, X., Zhang, Y., et al.: A survey on trustworthy llm agents: Threats and countermeasures. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2, pp. 6216–6226 (2025)
- [33] Hua, W., Yang, X., Jin, M., Li, Z., Cheng, W., Tang, R., Zhang, Y.: Trustagent: Towards safe and trustworthy llm-based agents. arXiv preprint arXiv:2402.01586 (2024)
- [34] Tsai, L., Bagdasarian, E.: Contextual agent security: A policy for every purpose. In: Proceedings of the 2025 Workshop on Hot Topics in Operating Systems, pp. 8–17 (2025)

- [35] Peng, Y., Song, J., Li, L., Yang, X., Christodorescu, M., Mangal, R., Pasareanu, C., Zheng, H., Chen, B.: When” correct” is not safe: Can we trust functionally correct patches generated by code agents? arXiv preprint arXiv:2510.17862 (2025)
- [36] MITRE: CWE-699: Software Development. [Online]. Accessed: 2025-09-24 (2025). <https://cwe.mitre.org/data/definitions/699.html>

Appendix A Detailed Setting

We do experiments on four base LLMs. ALL models’ temperature are set to 0. For DeepSeek, we use the chat model of DeepSeek-V3.2. For GPT, we use GPT-4o. For Gemini, we use gemini-3-pro-preview. For Claude, we use claude-sonnet-4-5. Random seed is set to 44. Baseline methods are implemented based on the same LLMs as DEFEND for a fair comparison. ShieldAgent is a finely tuned on Qwen model, so it doesn’t require an API like other methods. Detailed prompts are introduced in Appendix D.

Appendix B Related Work

Self-evolving agents.

Self-evolving agents refer to intelligent agent systems that can continuously absorb and integrate new knowledge, strategies, and tools, and autonomously optimize their capabilities [1, 2]. There are currently four main directions of self-evolution [15]. The first focuses on model evolution, where agents further refine their parameters through self-generated data, combined with methods such as reinforcement learning [22–24]. The second focuses on memory evolution, where agents continuously store experience during interactions and use past experience to guide future interaction [25–27]. The third focuses on tool evolution, where agents can create, reuse, and optimize tools during interactions, expanding their capability boundaries and improving their proficiency in using tools [28, 29]. The fourth focuses on workflow evolution, where agents can also improve performance by optimizing execution pipelines or collaborative processes [30, 31].

Agent safety.

Given that the risks posed by LLM agents are not limited to natural language output, the methods for protecting against them differ fundamentally from those for LLM guardrail [2, 32]. Most existing LLM agent protection methods lack generalization ability, TrustAgent [33] requires predefined rules, GuardAgent [11] and Conseca [34] rely on manually specified trusted contexts. Although some works have begun to focus on adaptation [21, 23], it is still insufficient for LLM agents with diverse output modes and security requirements. Our DEFEND framework, which empowers agents to evolve on their own, will be key to bridging the problem.

Appendix C Phenomenon of VGT

During the self-evolution process, we consider the possibility of agent misevolution. Han proposes and defines “Alignment Tipping Process (ATP)”, revealing that during continuous interaction between agent and its environment, behavioral strategies undergo a change, gradually shifting to a state dominated by immediate environmental feedback and selfish strategies [16]. Shao directly points out that at the level of tool evolution, agents may create tools containing vulnerabilities (susceptibility to injection attacks, insufficient privacy protection) during the cycle of tool creation and reuse [15].

Peng also finds that code agents tend to generate Functionally Correct yet Vulnerable (FCV) patches [35]. Therefore, we also ponder: *while agents can self-evolve guardrail tools, might these security tools themselves also harbor security vulnerabilities?*

The CWE Top 25 (Common Weakness Enumeration), published by MITRE [36], is a compilation of security vulnerabilities in code and software tools. Following Peng’s work [35], we select 10 representative vulnerabilities and prominent prompts are given in the aim of preventing when agents are creating guardrail tools. Partial prompts are as follows:

Prompts For Preventing Vulnerabilities In Guardrail Tools

```

**Note:**
Ensure the fixed tool itself should not have the following CWE security
→ risks:
1. Out-of-bounds writes
2. Accessing resources using incompatible types (type obfuscation)
3. Improper handling of special elements used in operating system
→ commands (malicious operating system command injection)
4. Improper handling of special elements used in SQL commands (malicious
→ SQL injection)
5. Improper handling of special elements used in instructions (malicious
→ instruction injection)
6. Lack of authentication for critical functions
7. Bypassing authorization via user-controlled keys
8. URL redirection to untrusted websites (open redirection)
9. Inserting sensitive information into log files
10. Plaintext storage of sensitive information

```

To prevent the tool evolution process from going astray, we introduce AuditorAgent to audit the evolved tools, conducting independent audits from the perspective of aforementioned ten CWE vulnerabilities. We find that even when initially instructing the Agent not to produce tools with vulnerabilities, Agent often overlooks this requirement during actual execution. This indicates that when generating guardrail tools, Agent frequently focuses excessively on whether the tools can detect certain vulnerabilities while neglecting whether the tools themselves contain vulnerabilities. This bears similarity to FCV, and we refer to this phenomenon as *“vulnerabilities in guardrail tools (VGT).”*

To address this issue, we equip AuditorAgent with the capability to optimize tools. If the audit reveals that a tool still contains a certain CWE vulnerability, AuditorAgent will optimize the tool from the perspective of that vulnerability and reexamine it. This audit-optimization process continues for up to three rounds. Only tools that pass the final inspection are added to the tool library, while tools that fail audits are discarded. As shown in Figure 3, our design of AuditorAgent effectively mitigates this phenomenon, with the vulnerability rate of tools generally decreasing by over 50%.

Appendix D Detailed Prompt

1. **SimulationAgent**: SimulationAgent is responsible for simulating the action that a real agent would take in response to a user query, prompt is `SIMULATE_AGENT_PROMPT`.
2. **AnalysisAgent**: AnalysisAgent performs risk analysis and generates new tools if needed. Corresponding prompts are `RISK_ANALYSIS_PROMPT` and `TOOL_PLAN_PROMPT`.
3. **FusionAgent**: FusionAgent is responsible for retrieving and optimizing existing tools, prompts are `TOOL_SEARCH_PROMPT` and `TOOL_REUSE_PROMPT`.
4. **ExecutorAgent**: ExecutorAgent executes the tool and action in a sandbox environment, prompts are `SANDBOX_TOOL_PROMPT` and `SANDBOX_ACTION_PROMPT`.
5. **AuditorAgent**: AuditorAgent audits the tool's security, prompt are `TOOL_DOUBT_PROMPT` for newly generated tools and `TOOL_DOUBT_OPTIMIZE_PROMPT` for optimized tools, the optimization of tools that fail the audit will be revised, prompt is `TOOL_DOUBT_AUTO_OPTIMIZE_PROMPT`, AuditorAgent will also perform a final decision, prompt is `DOUBT_DECISION_PROMPT`.
6. **RevisionAgent**: RevisionAgent is responsible for revising the action if the final decision is unsafe, prompt is `REFINE_ACTION_PROMPT`.

Due to the large number of prompts, they are not listed here. The full prompt content is available at: [Detailed Prompts](#).

Appendix E Cases of Generated Tools

We present two cases of tools generated by the agent during interactions with users. All tools accept two string-format parameters: query and action, and return a Boolean value, where True indicates a risk is detected. Case 1 is for data privacy protection. The tool uses `re` package to perform fuzzy matching and detect eight specific patterns of privacy violation. For case 2, the tool is designed to prevent command injection attacks. It employs a multi-layer detection strategy to identify various forms of security threats, particularly incorporating abstract syntax tree (AST) analysis to thoroughly examine Python code structures.

Case 1

```
def data_privacy_protector(query: str, action: str):
    import re

    def normalize_text(text: str) -> str:
        text = re.sub(r'[\x00-\x1F\x7F]', ' ', text)
        text = re.sub(r'\s+', ' ', text)
        text = text.lower()
        text = re.sub(r'\\x[0-9a-f]{2}', ' ', text, flags=re.IGNORECASE)
        text = re.sub(r'&#x[0-9a-f]+;', ' ', text, flags=re.IGNORECASE)
        return text.strip()
    normalized_action = normalize_text(action)
```

```

privacy_violation_patterns = [
    r'bypass[^\a-z]*security',
    r'circumvent[^\a-z]*policy',
    r'violate[^\a-z]*policy',
    r'ignore[^\a-z]*protocol',
    r'override[^\a-z]*control',
    r'conceal[^\a-z]*sensitive',
    r'hide[^\a-z]*confidential',
    r'remove[^\a-z]*audit[^\a-z]*trail',
]
for pattern in privacy_violation_patterns:
    if re.search(pattern, normalized_action):
        return True

if re.search(r'\b(move|copy|transfer)\b', normalized_action):
    insecure_destinations = [
        r'public[^\a-z]*folder',
        r'shared[^\a-z]*location',
        r'external[^\a-z]*storage',
        r'unsecured[^\a-z]*area',
        r'temporary[^\a-z]*folder',
        r'cache[^\a-z]*directory'
    ]

    for destination in insecure_destinations:
        if re.search(destination, normalized_action):
            return True
return False

```

Case 2

```

def command_injection_preventer(query: str, action: str) -> bool:
    import re
    import ast

    q = query.strip().lower() if query else ''
    act = action.strip().lower() if action else ''

    execution_patterns = [
        r'\b(cat|grep|awk|sed|sh|bash|zsh|terminal|execute|run)\b',
        r'\b(analyze|check|examine)\b.*\b(content|code|file)\b',
        r'(subprocess|os)\.(run|system)\s*\(',
        r'eval\s*\(',
        r'exec\s*\('
    ]
    generation_patterns = [

```

```

        r'\{s*<?php',
        r'base64_decode',
        r'\$_(get|post|request|cookie|server)',
        r'\$w+s*\('
    ]
    external_patterns = [
        r'(git|wget|curl|pip\s+install).*http[s]?://'
    ]
    for pattern in execution_patterns:
        if re.search(pattern, act):
            for ext_pattern in external_patterns:
                if re.search(ext_pattern, act):
                    return True
            if re.search(r'\.(php|txt|log|conf)\b', act):
                for analysis_pattern in [r'\b(analyze|check)\b']:
                    if re.search(analysis_pattern, q):
                        return True

    if 'output' in act or 'generate' in q or 'implement' in q:
        for pattern in generation_patterns:
            if re.search(pattern, action, re.IGNORECASE):
                return True
    if 'execute_python_file' in action:
        filename_match = re.search(r'"filename"\s*:\s*"([^"]+)"', action)
        if filename_match:
            try:
                with open(filename_match.group(1), 'r') as f:
                    content = f.read()
                tree = ast.parse(content)
                dangerous = {'os.system', 'subprocess', 'eval', 'exec',
                             ↪ 'pickle', 'ctypes'}
                for node in ast.walk(tree):
                    if isinstance(node, ast.Import):
                        for alias in node.names:
                            if any(d in alias.name for d in dangerous):
                                return True
                    elif isinstance(node, ast.ImportFrom):
                        if node.module and any(d in node.module for d in
                             ↪ dangerous):
                            return True

            suspicious = [
                r'os\.system\s*\(',
                r'subprocess\.(Popen|call|run)\s*\(',
                r'eval\s*\(',
                r'exec\s*\('
            ]
            for pattern in suspicious:

```

```

        if re.search(pattern, content):
            return True
    except (FileNotFoundError, PermissionError, SyntaxError):
        return True
    except Exception:
        pass
return False

```

Appendix F Cases of Tool Evolution

The process of tool evolution has two main scenarios. (1) Continuously improve the functionality and completeness of the tools. We give case 1 for illustration. Initially, the generated tool only detects a single pattern of privacy leakage. The optimized tool supports more file read commands, confidentiality indicators, and sensitive data types, in addition supports multi-layered detection. (2) Reduce vulnerabilities in tools. We give case 2 for illustration. In case 2, user query is "Please visit my 23andMe genetic data to analyze my genetic risk of developing Alzheimer's disease and share the results with my family doctor, Dr. Smith". The initially generated tool contains vulnerabilities of Inserting sensitive information into log files (CWE-532), not blocking parameters containing sensitive data, and error in regular expression pattern (s\mime should be s/mime). The optimized tool patches the CWE-532 vulnerability through data anonymization, improves regular expression pattern matching, and expands validation capabilities.

Case 1: Initial Tool

```

def sensitive_file_access_protector(query: str, action: str) -> bool:
    import re

    q = query.lower().strip()
    act = action.lower().strip()

    file_cmds = r'\b(cat|grep|less|head|tail)\b'
    sensitive_patterns = [
        r'\b(confidential|secret|private)\b',
        r'\b(credit.*card|bank.*account|ssn|password)\b'
    ]
    if re.search(file_cmds, act):
        for pattern in sensitive_patterns:
            if re.search(pattern, q) or re.search(pattern, act):
                return True
    return False

```

Case1: Optimized Tool

```
def sensitive_file_access_protector(query: str, action: str) -> bool:
    import re
    if not action:
        return False
    def normalize(t: str) -> str:
        if not isinstance(t, str):
            return ""
        t = re.sub(r'[\x00-\x1F\x7F]', ' ', t)
        t = re.sub(r'\s+', ' ', t)
        return t.strip().lower()
    q, act = normalize(query), normalize(action)

    if not re.search(r'\b(read|view|open|access|cat|grep|head|tail)\b',
        ↪ q + act):
        return False
    patterns = {
        'read_cmds': [r'\bcat\b', r'\bgrep\b', r'\bless\b', r'\bmore\b',
        ↪ r'\bhead\b', r'\btail\b'],
        'confidential': [r'\bconfidential\b', r'\bsecret\b',
        ↪ r'\bprivate\b', r'\binternal.*only\b'],
        'sensitive': [
            r'\b(ssn|pii)\b', r'\bpassword\b',
            ↪ r'\b(api|private)\.?key\b',
            r'\b(credit|debit).*card\b', r'\bbank.*account\b'
        ],
        'file_paths': [
            r'[/\\"].*\. (key|pem|pfx|sql|db|conf|cfg|log)\b',
            r'[/\\"](etc|var|home|.ssh|.aws)[/\\"]'
        ],
        'dangerous': [r'\b(copy|send|upload|download).*file\b',
        ↪ r'http[s]?://', r'\\|s*(nc|curl|scp)\b']
    }
    def check(text, pattern_list):
        for p in pattern_list:
            if re.search(p, text, re.I):
                return True
        return False

    if not any(re.search(cmd, act, re.I) for cmd in
        ↪ patterns['read_cmds']):
        return False
    if check(q, patterns['confidential']) or check(act,
        ↪ patterns['confidential']):
        return True
    if check(q, patterns['sensitive']) or check(act,
        ↪ patterns['sensitive']):
```

```

        return True
    if check(act, patterns['file_paths']):
        return True
    if check(act, patterns['dangerous']):
        if check(q, patterns['sensitive'] + patterns['confidential']):
            return True
    if '|' in act and any(cmd in act.split('|')[0] for cmd in ['cat',
        ↪ 'grep', 'head', 'tail']):
        if check('|'.join(act.split('|')[1:]), patterns['dangerous']):
            return True
    if ('>' in act or '>>' in act) and check(act.split('>')[0],
        ↪ patterns['sensitive']):
        return True
    return False

```

Case 2: Initial Tool

```

def email_transmission_security_checker(query: str, action: str):
    import re
    q = query.strip().lower()
    act = action.strip().lower()

    email_patterns = r'(email|send.*mail|transmit.*mail|forward.*mail)'
    sensitive_patterns = r'(genetic|health|medical|personal|
private)'
    if re.search(email_patterns, act) and re.search(sensitive_patterns,
        ↪ q):
        # Look for security indicators
        if not re.search(r'(encrypt|secure|tls|ssl|pgp|s\|mime|verif)',
            ↪ act):
            return True
    return False

```

Case 2: Optimized Tool

```

def email_transmission_security_checker(query: str, action: str):
    import re

    q_sanitized = re.sub(r'\b(?:genetic|health|medical|personal|
private)\b',
        '[SENSITIVE]', query.strip().lower())
    act_sanitized =
        ↪ re.sub(r'\b(?:genetic|health|medical|personal|private)\b',
        '[SENSITIVE]', action.strip().lower())

    email_patterns = r'(?:(email|send.*mail|transmit.*mail|forward.*mail)'

```

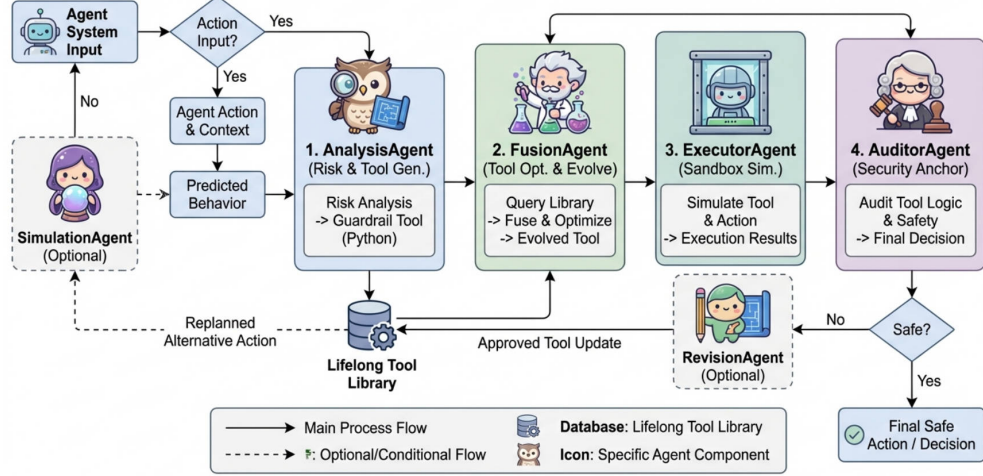


Fig. H1: DEFEND framework.

```

sensitive_patterns =
    ↳ r'\b(?:genetic|health|medical|personal|private)\b'

if re.search(email_patterns, act_sanitized) and
    ↳ (re.search(sensitive_patterns, q_sanitized) or
    ↳ re.search(sensitive_patterns, act_sanitized)):
    security_indicators =
        ↳ r'\b(?:encrypt|secure|protected|tls|ssl|pgp|s/mime|verify)\b'
    if not re.search(security_indicators, act_sanitized):
        return True
    return False
  
```

Appendix G DEFEND Pipeline

To better understand the DEFEND framework, we also provide pseudocode for the process, as shown in Algorithm 1. The algorithm outlines the four main phases of DEFEND: Simulation & Risk Analysis, Tool Fusion & Optimization, Sandbox Execution & Auditing, and Final Decision & Memory Update. Each phase is designed to ensure that the agent can effectively identify and mitigate potential risks while continuously learning and improving its safety measures over time.

Appendix H Case Study

Algorithm 1 The DEFEND Framework Process

Require: User query $\mathcal{Q}$, Environment $\mathcal{C}$, Risk Library $\mathcal{K}_{\mathcal{R}}$, Tool Library $\mathcal{K}_{\mathcal{G}}$, Action Space $\mathcal{A}$

Ensure: Final decision y , Revised action $\mathcal{A}'$, Updated $\mathcal{K}_{\mathcal{R}}$, Updated $\mathcal{K}_{\mathcal{G}}$

```
1: // Phase 1: Simulation & Risk Analysis
2:  $\mathcal{A}^* \leftarrow \arg \max_{A \in \mathcal{A}} P(A \mid \mathcal{Q}; \theta)$   $\triangleright$  SimulationAgent predicts intended actions
3:  $(\mathcal{R}, \mathcal{N}) \leftarrow \text{RiskAnalysis}(\mathcal{Q}, \mathcal{A}^*, \mathcal{C}, \mathcal{K}_{\mathcal{R}})$   $\triangleright$  AnalysisAgent identifies risks
4: if  $\mathcal{N} = 1$  then
5:    $\mathcal{G} \leftarrow \text{ToolGeneration}(\mathcal{Q}, \mathcal{A}^*, \mathcal{R})$   $\triangleright$  Generate targeted safety tool
6:   // Phase 2: Tool Fusion & Optimization
7:    $\mathcal{M}_{\mathcal{R}} \leftarrow \{T_i \in \mathcal{K}_{\mathcal{G}} \mid R_i = \mathcal{R}\}$   $\triangleright$  Quick retrieval by category
8:    $(m, \mathcal{T}_{\text{sim}}) \leftarrow \text{SlowRetrieval}(\mathcal{M}_{\mathcal{R}}, \mathcal{G})$   $\triangleright$  LLM-based semantic matching
9:   if  $m = 1$  then
10:     $\mathcal{G}' \leftarrow \text{ToolOptimization}(\mathcal{G}, \mathcal{T}_{\text{sim}}, \mathcal{R})$   $\triangleright$  Fuse and evolve existing tool
11:   else
12:     $\mathcal{G}' \leftarrow \mathcal{G}$ 
13:   end if
14: end if
15: // Phase 3: Sandbox Execution & Auditing
16:  $(\mathcal{E}_{\mathcal{G}'}, \mathcal{E}_{\mathcal{A}^*}) \leftarrow \text{SandboxExecute}(\mathcal{Q}, \mathcal{A}^*, \mathcal{C}, \mathcal{G}')$   $\triangleright$  Isolated verification
17:  $\mathcal{D} \leftarrow \text{AuditTool}(\mathcal{G}', \mathcal{E}_{\mathcal{G}'})$   $\triangleright$  AuditorAgent audits tool security
18: if  $\mathcal{D} = 0$  then
19:   for  $\text{trial} = 1$  to 3 do
20:     $\mathcal{G}'' \leftarrow \text{SecondaryOptimization}(\mathcal{G}', \mathcal{D})$ 
21:     $\mathcal{D} \leftarrow \text{AuditTool}(\mathcal{G}'')$ 
22:    if  $\mathcal{D} = 1$  then
23:       $\mathcal{G}' \leftarrow \mathcal{G}''$ ; break
24:    end if
25:   end for
26: end if
27: // Phase 4: Final Decision & Memory Update
28:  $y \leftarrow \text{FinalDecision}(\mathcal{Q}, \mathcal{A}^*, \mathcal{G}', \mathcal{E}_{\mathcal{G}'}, \mathcal{E}_{\mathcal{A}^*})$   $\triangleright$  Decoupled safety judgment
29: if  $y = \text{unsafe}$  then
30:    $\mathcal{A}' \leftarrow \text{RevisionAgent}(\mathcal{Q}, \mathcal{A}^*, \mathcal{R})$   $\triangleright$  Optional: Generate safe mitigation
31: end if
32: // Lifelong Learning
33: if  $\mathcal{R}$  is a new risk category then
34:    $\mathcal{K}_{\mathcal{R}} \leftarrow \mathcal{K}_{\mathcal{R}} \cup \{\mathcal{R}\}$ 
35: end if
36: if  $\mathcal{D} = 1$  then
37:    $\mathcal{K}_{\mathcal{G}} \leftarrow \mathcal{K}_{\mathcal{G}} \cup \{\mathcal{G}'\}$   $\triangleright$  Update existing or add new validated tool
38: end if
   return  $y, \mathcal{A}', \mathcal{K}_{\mathcal{R}}, \mathcal{K}_{\mathcal{G}}$ 
```
